

E-COMMERCE SALES & CUSTOMER ANALYTICS DATABASE

by Harsh Tripathi

Objective

For this project, I built a normalized MySQL database to simulate a real-world e-commerce business. I designed the schema, inserted sample transactional data, and wrote SQL queries ranging from basic to advanced to answer common business questions about customers, orders, and sales.

Database Schema

The database (ECOMMERCE) has 5 tables connected through primary and foreign keys:

Table	Columns
customers	customer_id (PK), customer_name, email, city, signup_date
products	product_id (PK), product_name, category, price
orders	order_id (PK), customer_id (FK), order_date, order_status
order_items	order_item_id (PK), order_id (FK), product_id (FK), quantity
payments	payment_id (PK), order_id (FK), payment_mode, payment_status, payment_date

Sample Data

Inserted 5 customers, 5 products, 5 orders, 7 order items, and 5 payment records to simulate realistic transactions (including a cancelled order and a failed payment).

Skills Demonstrated

- Database design and normalization
- Joins across multiple tables (inner join, left join)
- Aggregate functions: SUM, COUNT, AVG, MAX
- Window functions: RANK, DENSE_RANK, ROW_NUMBER
- Subqueries, GROUP BY, and HAVING clauses
- Solving real business questions using SQL

Queries and Results

Basic Level

Query 1: List all customers from Delhi

```
SELECT * FROM customers WHERE city = 'Delhi';
```

Result:

customer_id	customer_name	email	city	signup_date
1	Amit Sharma	amit@gmail.com	Delhi	2024-01-10

Query 2: Show all products with price > 5000

```
SELECT * FROM products WHERE price > 5000;
```

Result:

product_id	product_name	category	price
101	iPhone 15	Electronics	75000
102	Samsung TV	Electronics	45000
103	Nike Shoes	Fashion	6000

Query 3: Find total number of orders

```
SELECT COUNT (order_id) AS 'total order' FROM orders;
```

Result:

total order
5

Query 4: Display all delivered orders

```
SELECT * FROM orders WHERE order_status = 'Delivered';
```

Result:

order_id	customer_id	order_date	order_status
1001	1	2024-03-10	Delivered
1002	2	2024-03-12	Delivered
1004	3	2024-03-18	Delivered

Query 5: Count total customers city-wise

```
SELECT city, COUNT (customer_name) AS 'total customer' FROM customers GROUP BY city;
```

Result:

city	total customer
Bangalore	1
Delhi	1

city	total customer
Gurgaon	1
Mumbai	1
Pune	1

Query 6: Rank orders by status (DENSE_RANK)

```
SELECT *, DENSE_RANK () OVER (PARTITION BY order_status ORDER BY order_status) AS
order_status_rank
FROM orders;
```

Result:

order_id	customer_id	order_date	order_status	order_status_rank
1003	1	2024-03-15	Cancelled	1
1001	1	2024-03-10	Delivered	1
1002	2	2024-03-12	Delivered	1
1004	3	2024-03-18	Delivered	1
1005	4	2024-03-20	Pending	1

Query 7: Rank products by price within category (ROW_NUMBER)

```
SELECT product_name, category, price,
ROW_NUMBER () OVER (PARTITION BY category ORDER BY price) AS row_Num
FROM products;
```

Result:

product_name	category	price	row_Num
Laptop Bag	Accessories	2000	1
Bluetooth Headphones	Electronics	3000	1
Samsung TV	Electronics	45000	2
iPhone 15	Electronics	75000	3
Nike Shoes	Fashion	6000	1

Easy Level

Query 8: Find total orders placed by each customer

```
SELECT c.customer_name, COUNT(o.order_id) AS 'total_order' FROM customers AS c
JOIN orders AS o ON c.customer_id = o.customer_id
GROUP BY c.customer_name
ORDER BY total_order DESC;
```

Result:

customer_name	total_order
Amit Sharma	2

customer_name	total_order
Rohit Singh	1
Pooja Mehta	1
Neha Verma	1

Query 9: Find total sales amount per product

```
SELECT o.product_id, SUM(p.price) AS 'total_price' FROM products AS p
JOIN order_items AS o ON p.product_id = o.product_id
GROUP BY p.product_id
ORDER BY total_price;
```

Result:

product_id	total_price
105	3000
104	4000
103	12000
102	45000
101	75000

Query 10: Show customer name and their order date

```
SELECT c.customer_name, o.order_date FROM customers AS c
JOIN orders AS o ON c.customer_id = o.customer_id;
```

Result:

customer_name	order_date
Amit Sharma	2024-03-10
Neha Verma	2024-03-12
Amit Sharma	2024-03-15
Rohit Singh	2024-03-18
Pooja Mehta	2024-03-20

Query 11: Find most expensive product

```
SELECT * FROM products WHERE price = (SELECT MAX(price) FROM products);
```

Result:

product_id	product_name	category	price
101	iPhone 15	Electronics	75000

Query 12: Show all orders where payment failed

```
SELECT o.customer_id, o.order_date, o.order_status, p.payment_status FROM orders AS o
JOIN payments AS p ON o.order_id = p.order_id
```

```
WHERE p.payment_status = 'failed';
```

Result:

customer_id	order_date	order_status	payment_status
1	2024-03-15	Cancelled	Failed

Advanced Level (Interview Level)

Query 13: Find total revenue generated

```
SELECT o.order_status, SUM(p.price * oi.quantity) AS 'total revenue'
FROM products p
JOIN order_items AS oi ON p.product_id = oi.product_id
JOIN orders AS o ON o.order_id = oi.order_id
JOIN payments AS pmt ON pmt.order_id = o.order_id
WHERE o.order_status = 'Delivered' AND pmt.payment_status = 'Success';
```

Result:

order_status	total revenue
Delivered	136000

Query 14: Find customer who spent the most money

```
SELECT c.customer_name, SUM(p.price*oi.quantity) AS 'total_spent_money' FROM customers AS c
JOIN orders AS o ON c.customer_id = o.customer_id
JOIN order_items AS oi ON o.order_id = oi.order_id
JOIN products AS p ON p.product_id = oi.product_id
GROUP BY c.customer_id, c.customer_name ORDER BY total_spent_money DESC LIMIT 1;
```

Result:

customer_name	total_spent_money
Amit Sharma	85000

Query 15: Find top 3 selling products by quantity

```
SELECT p.product_name, SUM(oi.quantity) AS 'total_quantity' FROM products AS p
JOIN order_items AS oi ON p.product_id = oi.product_id
GROUP BY p.product_name ORDER BY total_quantity DESC LIMIT 3;
```

Result:

product_name	total_quantity
Laptop Bag	3
Nike Shoes	2
Bluetooth Headphones	2

Query 16: Find monthly sales report

```
SELECT MONTH(o.order_date) AS 'months', SUM(p.price*oi.quantity) AS 'total_sales' FROM
orders AS o
JOIN order_items AS oi ON o.order_id = oi.order_id
JOIN products AS p ON p.product_id = oi.product_id
GROUP BY months;
```

Result:

months	total_sales
3	144000

Query 17: Find customers who never placed any order

```
SELECT c.customer_id, c.customer_name FROM customers AS c
LEFT JOIN orders AS o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Result:

customer_id	customer_name
5	Rahul Kumar

Query 18: Find cancelled orders with payment status

```
SELECT o.order_status, pmt.payment_status FROM orders AS o
JOIN payments AS pmt ON o.order_id = pmt.order_id
WHERE o.order_status = 'cancelled';
```

Result:

order_status	payment_status
Cancelled	Failed

Query 19: Find average order value

```
SELECT AVG(p.price * oi.quantity) AS 'ave_price' FROM products AS p
JOIN order_items AS oi ON p.product_id = oi.product_id;
```

Result:

ave_price
20571.43

Query 20: Find orders with more than 1 item

```
SELECT o.order_id, SUM(oi.quantity) AS 'total_item' FROM orders AS o
JOIN order_items AS oi ON o.order_id = oi.order_id
GROUP BY o.order_id
HAVING SUM(oi.quantity) > 1;
```

Result:

order_id	total_item
1001	3
1004	3

Query 21: Find repeat customers

```
SELECT c.customer_name, COUNT(o.order_id) AS 'total_order', o.customer_id FROM customers AS c
JOIN orders AS o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
HAVING COUNT(o.order_id) > 1;
```

Result:

customer_name	total_order	customer_id
Amit Sharma	2	1

Query 22: Rank customers by total spending (subquery + RANK)

```
SELECT customer_id, customer_name, total_spending,
       RANK() OVER (ORDER BY total_spending DESC) AS rank_num
FROM (
  SELECT c.customer_id, c.customer_name, SUM(p.price * oi.quantity) AS total_spending
  FROM customers c
  JOIN orders o ON o.customer_id = c.customer_id
  JOIN order_items oi ON oi.order_id = o.order_id
  JOIN products p ON p.product_id = oi.product_id
  GROUP BY c.customer_id, c.customer_name
) t;
```

Result:

customer_id	customer_name	total_spending	rank_num
1	Amit Sharma	85000	1
2	Neha Verma	45000	2
3	Rohit Singh	12000	3
4	Pooja Mehta	2000	4

Analysis Summary

Looking at the results, I found that Amit Sharma is my top customer, both in number of orders and in total money spent (₹85,000). The iPhone 15 brings in the most revenue since it's the most expensive item, while the Laptop Bag actually sold the most units. Total revenue from delivered orders with successful payments came out to ₹136,000, and the average order value is around ₹20,571. Since all the orders happened in March 2024, the monthly sales query only gives one month of data right now.

Points of Strength

- I kept the tables normalized and connected them properly using primary and foreign keys.

- My revenue query only counts Delivered orders with Successful payments, so cancelled or pending orders don't inflate the numbers.
- I was able to move from simple queries to joins, aggregates, subqueries, and even window functions, which shows I practiced a good range of SQL.
- I handled edge cases correctly too, like finding customers with no orders using a LEFT JOIN and finding repeat customers using HAVING.

Conclusion

This project helped me practice designing a relational database from scratch and writing SQL queries to solve real business problems — from simple lookups to more advanced analytics like revenue calculation, customer ranking, and repeat customer identification.